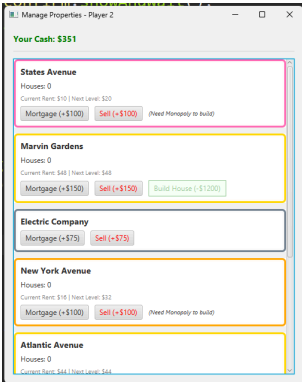




Click [HERE](#) for the Task Description

Manual Tests (add more rows as needed)

TEST #	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	PASS/FAIL	PROBLEM DESCRIPTION	FIXED?
1	Tile 17	Community Chest	Tile 17	Fail	The code is intended to assign each tile a suitable purpose. Some are buildings, for example. For special tiles, there is a special tile array to map all the non-property tiles. I had null for tile 17 as a placeholder. I forgot to update it, and that error was created.	Yes , the error was addressed in the array Special Tiles
2	Move to tile 18	Token moves to Tennessee Avenue	The token moves between New York Avenue and Tennessee Avenue	Fail	The token is the visual representation of a player's location on the board. In this case, the token landed between two properties. While the player knows what property they're on because of the right side information, this is still unideal due to the conflict in signals. This error was caused by the offset I applied to the tokens to make sure that if two tokens landed on the same property, they would not overlap. To fix this, I will reduce the overlap.	Fail , it helped sometimes. However, overall, the tokens still occasionally land between tiles. To compensate, the user can look to the right side panel to get info about the tile they landed on
3	Gets a monopoly, player tries to buy a house for Ventor Avenue.	Charged 150 to upgrade Ventor Avenue with a house.	 In this image, the cost to upgrade a house on Ventura Avenue is 1150. That's the hotel rent, but the cost to upgrade it with one house is 150.	Fail	Currently, the program is misreading properties.txt. It expects 7 rents, but in actuality, currently, my properties.txt only has 6. To address this issue, I must update the properties.txt.	
4	Player lands on a mortgaged property.	The player does not have to pay, as they have landed on a property that has been mortgaged out to the bank.	Sometimes, the rent was still being charged despite my property being mortgaged.	Fail	My rent calculation logic did not consistently check for the mortgaged boolean value before charging rent, particularly when rent was calculated immediately after landing.	Yes , I added an explicit mortgage check in the rent calculation logic so that mortgaged properties would never charge rent.
5	After failing to pay fees, a player chooses to go bankrupt.	After a player declares bankruptcy, their properties should be given to the creditor, and if	After a bankruptcy, a creditor sometimes didn't get a monopoly even if they had all of the coloured property.	Fail	This was caused by the fact that my monopoly check logic only triggers when properties are purchased, not when ownership is changed through bankruptcy.	Yes , to fix this, I reused the monopol



		that means the creditor then gains a monopoly on some properties, that should be updated automatically.				y-checkin g method after property transfers to ensure monopoli es would be rechecke d wheneve r ownershi p changed hands.
6	Player rolls the dice once.	The player should be unable to roll the dice a second time.	The player was able to roll the dice multiple times, as the roll dice button was not disabled.	Fail	The roll button was not disabled, allowing users to roll multiple times.	Yes , I disabled the roll button as soon as a roll was complete d and re-enable d it after a turn was complete .
7	A user passes Go	The user receives \$200.	The user received \$200	Pass		
8	Purchase Property	The user should be able to see the property by clicking my properties, and checking the new window.	The player is able to see the new property by clicking on my property.	Pass		
9	A user gets a monopoly over a colour set of properties.	The user should be able to buy houses.	The user is able to buy houses.	Pass		

If you have JUnit Tests, include them in your [repository](#).

Concerns <i>Areas that need improvement</i>	Criteria	Strengths <i>Areas that exceed expectations</i>	Level
A4.2 Develop and implement a formal testing plan for a software project to ensure correctness			
	My test plan/execution log is complete and covers both valid and invalid inputs, including boundary cases. The execution of my test plan authentically demonstrates the debugging process needed to create the final program. Weight: 10		